

Re-architecture of the DisloCluster 0-D Solver and of the 0-D $\leftrightarrow$ 3-D Coupling

DisloCluster development note

8 August 2026

Abstract

Three changes were made to the DisloCluster 0-D side and to its coupling with the spatially resolved code. (i) The right-hand side was moved into a single scalar-type-templated core so that the residual and an *exact* Jacobian, obtained by forward-mode automatic differentiation, are two instantiations of one definition; SUNDIALS' difference-quotient Jacobian is thereby retired. (ii) The six point-defect conservation accumulators were recognized as pure quadratures and moved out of the implicit system into CVODES quadrature variables, shrinking the Newton block from 19×19 to 13×13 standalone and to 9×9 under the operator split. (iii) The 0-D $\rightarrow$ 3-D handoff, which previously collapsed the marched state to four scalars, was replaced by a genuine per-node field exchange through MoDELib3's own cluster-dynamics configuration block.

Measurement then redirected the effort: at $n = 19$ the dense linear algebra was never the bottleneck. Profiling with the integration window collapsed to zero showed that per-case construction and destruction of SUNDIALS objects consumed 1.31 s of thread time against 0.2 s of actual solving in a 256-point coupling substep. Reusing one solver workspace per OpenMP thread cut the end-to-end march wall time by a factor of 1.67 at $q = 1024$ quadrature points — more than the two originally requested changes combined. The report states all three outcomes as measured, including the two that did not pay off in wall time.

Two further sections document the coupling in operation. Section 6.6 measures the full 1–26 dpa march on the 1 μm single crystal in three configurations, separating the genuine solver speedup ($1.95\times$ at identical work) from the cost of resolving a field on 24 115 nodes instead of averaging 25 line points to four scalars. Section 7 characterizes MoDELib3's own nodal immobile integrator against the CVODE march — correcting, along the way, the common description of it as explicit Euler: it is a first-order semi-implicit scheme, and it has to be, because the measured stiffness puts its step 46 000 times beyond the forward-Euler limit. Section 8 then traces where each scheme came from — the immobile field does not exist at all in the upstream MoDELib2-NNL — and draws out the maintenance consequence of the two different routes.

Contents

1	Starting point	1
2	Structural observation: the accumulators are pure quadratures	2
3	Implementation	2
3.1	One templated core, two instantiations	2
3.2	Reduced implicit block	3
3.3	Reusable per-thread solver workspace	3
3.4	Interface	3
4	Verification	4
4.1	The AD Jacobian	4
4.2	Bit-identity of the untouched path	4

5	Results	4
5.1	Standalone 0-D run	5
5.2	Operator-split coupling march	5
5.3	Where the improvement actually came from	6
5.4	A step-size pathology the reduction cures	6
5.5	The quadrature memo, and why the reduction does not win on time	7
6	The 0-D $\leftrightarrow$ 3-D field handoff	7
6.1	What was wrong	7
6.2	What MoDELib3 actually offers	7
6.3	The mapping	7
6.4	Verification on the 3-D verification case	8
6.5	Notebook wiring	8
6.6	Measured cost of the field upgrade	9
7	Integrator comparison: MoDELib’s nodal update versus the CVODE march	10
7.1	What MoDELib actually does	10
7.2	Test method	10
7.3	Stability	10
7.4	Accuracy and speed	11
7.5	Consequence for the coupling	12
8	Provenance: how the two schemes came to be	12
8.1	The standalone scheme: from absent, to explicit, to semi-implicit	13
8.2	The coupled march scheme: outward from the 0-D solver	14
8.3	Two routes to the same conclusion	14
9	Summary and recommended next steps	14

1 Starting point

The 0-D model integrates $N_{\text{eq}} = 19$ ordinary differential equations: 12 physical species, 6 point-defect conservation accumulators, and the evolving network dislocation density ρ_N . In the operator-split coupling the four mobile species $\mathbf{C}_M = [C_v, C_i, C_{2i}, C_{3i}]$ are frozen at the value supplied by the 3-D fast solve (`freeze_mobile=1`), and the remaining state is marched independently at every quadrature point, one case per point in an OpenMP batch subprocess.

Before this work the solver had the following properties.

- **No Jacobian was supplied.** `CvodeSetJacFn` was never called, so SUNDIALS formed the 19×19 dense Jacobian by difference quotients, costing 19 extra residual evaluations per Jacobian setup.
- **All 19 equations were implicit.** The accumulators were carried as ODE state, entered the Newton system and the error test, and were factorized in every LU.
- **Every case built its own solver.** `integrate_one` created and destroyed a `SUNContext`, state vector, dense matrix, linear solver and CVODE memory per invocation — i.e. once per quadrature point per substep.
- **The 0-D $\rightarrow$ 3-D carrier was four scalars.** The driver averaged the marched state over all points and wrote one number density and one mean radius per loop family into MoDELib’s microstructure generator inputs.

2 Structural observation: the accumulators are pure quadratures

Rows 12–17 of the right-hand side are written but columns 12–17 are never read: no equation in the system depends on an accumulator value. Inspection of `rate_equations.cpp` confirms that `y[12..17]` appears nowhere on the right of any assignment. The Jacobian therefore has the block structure

$$J = \begin{pmatrix} J_{ss} & 0 \\ J_{as} & 0 \end{pmatrix}, \quad s = \{\text{species}, \rho_N\}, \quad a = \{\text{accumulators}\}, \quad (1)$$

so the accumulators are quadratures of the state trajectory, $\mathbf{a}(t) = \int \mathbf{g}(\mathbf{y}_s(\tau)) d\tau$, and belong in CVODES’ quadrature module rather than in the Newton system.

Combining this with the operator split, which freezes the four mobile species, gives the implicit block dimensions of Table 1.

Table 1: Dimension of the implicit block actually factorized.

Configuration	Newton block n	Dense LU $\propto n^3$	Relative LU cost
Legacy (all 19 implicit)	19	6859	1.00
Reduced, standalone	13	2197	0.32
Reduced, <code>freeze_mobile=1</code>	9	729	0.11

3 Implementation

3.1 One templated core, two instantiations

The physics was moved verbatim from `rate_equations.cpp` into `rate_equations_core.h` as

```
template <class T> void rhs_core(const T* y, T* ydot, const Parameters& P)
```

instantiated with `T = double` for the residual and with `T = Dual<N>` (`dual.h`) for the Jacobian. Forward-mode automatic differentiation evaluates value and all N directional derivatives in a *single* sweep: `sqrt`, `exp` and `cbt` are evaluated once and their derivatives propagated by a multiply, rather than the N complete residual evaluations a difference quotient needs.

Two properties motivated AD over a hand-written analytic Jacobian. First, exactness: there is no perturbation size to choose and no truncation error. Second, and more important for maintenance, the Jacobian cannot drift out of sync with the residual — hand-differentiating the coalescence Avrami gates and the smoothstep minimum-size gate would have created a second copy of the physics to keep aligned, on a code base that already maintains a Python mirror.

Bit-identity. Floating-point addition is not associative, so the transcription preserved the exact grouping of every expression. Where the original wrote `k1 * k2 * C * D`, meaning $((k_1 k_2)C)D$, the templated core hoists $k_1 k_2$ into a named `double` and keeps the remaining order. This was verified rather than asserted: a reference binary was built from the git-HEAD sources and the outputs compared byte for byte (Section 5).

3.2 Reduced implicit block

With `-reduced=1` the state vector holds only the implicit components and the accumulators are advanced by `CVodeQuadInit`. Quadrature error control is left at its default (off), so the accumulators ride the step sequence the state equations demand instead of constraining it. The 19-column output contract is unchanged: each row is reassembled from the reduced state, the frozen mobile values and the quadrature vector.

3.3 Reusable per-thread solver workspace

This change was not planned; it was the result of profiling. Running a 256-case frozen-mobile batch with the integration window collapsed to $t_{\text{end}} = t_{\text{begin}}(1 + 10^{-12})$ — so that only process spawn, case-file parsing and output formatting remain — gave the decomposition in Table 2.

Table 2: Cost decomposition of one 256-case coupling substep, before the workspace change. “cpu-solve” is the sum of per-case integration wall times over all threads.

Configuration	wall [s]	cpu-solve [s]
Null integration (setup/teardown only)	0.172	1.310
1 dpa, 19×19 difference-quotient	0.136	1.608
1 dpa, 19×19 AD	0.129	1.491

Constructing and destroying the SUNDIALS objects cost 1.31 s of thread time against roughly 0.2 s of genuine integration: the per-case setup was consuming about six times what the ODE solve did. A `Workspace` is now created once per OpenMP thread and reused across every case that thread handles via `CVodeReInit` / `CVodeQuadReInit`, rebuilt only when a case needs a different shape. In a 1024-point march this reduces 1024 solver constructions per substep to 24, one per thread.

3.4 Interface

Three flags were added; both numerical flags default to *off* so that a standalone 0-D run remains bit-identical to the previous solver. `modelib_coupling.build_immobile_cases` turns them on for the march, where they are measurably better.

Flag	Effect
<code>-reduced=1</code>	Accumulators as CVODES quadrature; Newton block 13 (or 9 with <code>freeze_mobile</code>). CVODE only.
<code>-analytic_jac=1</code>	Exact dense Jacobian by forward AD. Dense linear solver only.
<code>-stats=1</code>	Emit a <code># STATS</code> comment line with the integrator counters. Non-numeric, so the existing Python row parsers skip it.

4 Verification

4.1 The AD Jacobian

A standalone tool (`jac_check.cpp`) compares the AD Jacobian against a central-difference Jacobian of the *same* core, differencing column j with a relative step $h_j = \varepsilon_{\text{rel}}|y_j|$ because the state spans some 34 decades. It was run at 20 states sampled along a converged trajectory from 10^{-8} to 100 dpa.

Two classes of entry are excluded, for reasons belonging to the finite difference and not to the AD:

- **FD-unresolvable entries.** A central difference can only see an entry whose contribution $J_{ij}h_j$ rises above the floating-point granularity of f_i . At the initial condition, dC_i/dt is pinned near $G_i \approx 10^{-7}$ while its C_v derivative moves it by $\sim 10^{-28}$, i.e. 10^{-5} of one unit in the last place. The difference quotient returns exactly zero there at *any* step size; AD returns the exact value. Entries are compared only where the signal exceeds 100 ULP. Note that this is precisely the blindness SUNDIALS’ difference-quotient Jacobian suffers, so the AD Jacobian is strictly more informative in this region.

- **Components on the concentration floor**, where `f1()` makes the residual non-smooth: AD returns the one-sided derivative while a central difference straddles the kink.

The result is in Table 3. The discrepancy falls by two decades per decade of ε_{rel} across the usable range, the $O(h^2)$ signature of a correct derivative with the finite difference’s own truncation error dominating.

Table 3: AD versus central-difference Jacobian, worst row-relative discrepancy over all compared entries. Representative states from the 20 tested.

Dose [dpa]	$\varepsilon_{\text{rel}} = 10^{-5}$	10^{-6}	10^{-7}	entries compared
1.0×10^{-8}	6.50×10^{-12}	8.84×10^{-11}	1.24×10^{-9}	6
1.3×10^{-6}	8.49×10^{-8}	7.96×10^{-7}	2.05×10^{-6}	108
1.6×10^{-4}	8.29×10^{-8}	9.97×10^{-7}	1.83×10^{-5}	117
2.1×10^{-2}	4.40×10^{-7}	4.71×10^{-6}	5.16×10^{-5}	133
7.8×10^{-1}	1.55×10^{-7}	9.30×10^{-7}	9.92×10^{-6}	133
8.9×10^0	3.09×10^{-9}	4.28×10^{-8}	1.30×10^{-6}	107
1.0×10^2	8.04×10^{-9}	5.17×10^{-8}	5.71×10^{-7}	107
Worst over all 20 states (best ε per state): 4.40×10^{-7} ; states failing at every ε : 0/20.				

4.2 Bit-identity of the untouched path

The legacy configuration of the rebuilt solver reproduces the git-HEAD reference binary *byte for byte* — 1000 output rows $\times$ 19 columns in the standalone run, and the endpoint state of all 256 points $\times$ 5 substeps in the coupling march. The refactor into a templated core therefore changed no arithmetic.

5 Results

All runs use `input/Zr_input_parameters.xlsx` at $T = 573$ K, $G = 10^{-7}$ dpa/s, on 24 cores. Because wall-clock times on a shared machine are noisy, exact operation counts are reported alongside and are the primary evidence:

$$n_{\text{core}} = \text{residual-core evaluations} = \text{nfe} + \text{nfeLS} + \text{nqmiss},$$

$$n_{\text{LU}} = \text{dense factorizations (nsetups)}, \quad \text{LU flops} \propto n_{\text{LU}} n^3/3.$$

5.1 Standalone 0-D run

$t = 10^{-1} \dots 10^9$ s (100 dpa), 1000 output points, $\text{rtol} = 10^{-6}$, $\text{atol} = 10^{-20}$. Accuracy is measured against a tight reference ($\text{rtol} = 10^{-10}$, $\text{atol} = 10^{-24}$), compared per column only where the reference exceeds 10^{-8} of that column’s own peak — a plain elementwise relative error is meaningless on a state spanning 34 decades whose early transient sits below atol .

The AD Jacobian removes 32% of the residual evaluations ($6129 \rightarrow 4171$) and 11% of the steps. The reduction cuts LU work by 71% but *adds* residual evaluations, for the reason given in Section 5.5. All four configurations sit within a factor of three of each other in accuracy, at the level expected from different step sequences at the same tolerance; none is meaningfully more accurate than another.

5.2 Operator-split coupling march

A 5 dpa dose step in 5 substeps, seeded at 1 dpa, with the mobile field depleted toward a sink across the point set. Table 6 gives $q = 1024$; the $q = 64$ and $q = 256$ rows scale proportionally.

Table 4: Standalone 0-D run. “bit=ref” is byte-for-byte agreement with the git-HEAD binary.

Configuration	wall [ms]	solve [ms]	nst	n_{core}	n_{LU}	LU flops	bit=ref
git HEAD 19×19 DQ	70.7	—	—	—	—	—	—
Baseline 19×19 DQ	69.3	46.7	2797	6129	466	1.07×10^6	yes
AD Jac. 19×19 AD	65.5	46.9	2483	4171	429	9.81×10^5	no
Reduced 13×13 DQ	64.5	42.3	2564	7823	423	3.10×10^5	no
Reduced + AD 13×13	63.4	43.7	2811	7415	458	3.35×10^5	no

Table 5: Standalone accuracy against the tight reference (max relative error).

Configuration	species	accumulators	ρ_N
git HEAD / baseline 19×19 DQ	1.41×10^{-4}	2.15×10^{-5}	1.60×10^{-5}
AD Jac. 19×19	2.18×10^{-4}	4.90×10^{-5}	3.16×10^{-5}
Reduced 13×13 DQ	2.44×10^{-4}	5.25×10^{-5}	3.30×10^{-5}
Reduced + AD 13×13	8.79×10^{-5}	2.18×10^{-5}	1.30×10^{-5}

Agreement with the git-HEAD march is 1.3×10^{-5} or better in every species at every q — the expected splitting-consistent difference between step sequences, not a change of answer.

5.3 Where the improvement actually came from

Stated plainly, and against the expectation that motivated the work:

- **Workspace reuse is the whole wall-clock gain:** $1.67\times$ at $q = 1024$, and it grows with q because the per-case setup it removes scales with the point count while the per-thread construction does not.
- **The AD Jacobian delivers its target metric:** residual evaluations fall by 33% ($366\,170 \rightarrow 243\,682$) and difference-quotient evaluations go to zero. Wall time is neutral, because one Dual<19> sweep costs about what the 19 scalar evaluations it replaces cost. Its durable value is exactness, the elimination of the FD-blind entries of Table 3, and a Jacobian that cannot fall out of step with the residual.
- **The 9×9 reduction delivers its target metric — a 92% cut in LU flops — but LU was never the bottleneck at $n = 19$.** It also reduces the step count by 18%. It nonetheless loses slightly in wall time, because of the quadrature cost analyzed next.

Table 6: Coupling march, $q = 1024$ quadrature points, 5 substeps. “warn” counts CVODE roundoff warnings (see Section 5.4); error is against the git-HEAD result.

Configuration	wall [s]	cpu-solve [s]	nst	n_{core}	n_{LU}	LU flops	warn
git HEAD 19×19 DQ	2.379	—	—	—	—	—	2048
Reuse 19×19 DQ	1.424	12.03	175771	366170	69860	1.60×10^8	2048
Reuse + AD 19×19	1.435	12.98	175890	243682	69921	1.60×10^8	2048
Reduced 9×9 DQ	1.568	13.99	143651	400404	54959	1.34×10^7	0
Reduced + AD 9×9	1.572	13.68	143654	349626	54957	1.34×10^7	0

Table 7: March wall time versus point count. Speedup is against the git-HEAD binary at the same q .

Configuration	$q = 64$		$q = 256$		$q = 1024$	
	[s]	speedup	[s]	speedup	[s]	speedup
git HEAD	0.254	1.00	0.692	1.00	2.379	1.00
Reuse 19×19 DQ	0.214	1.19	0.482	1.44	1.424	1.67
Reuse + AD 19×19	0.211	1.20	0.468	1.48	1.435	1.66
Reduced + AD 9×9	0.217	1.17	0.502	1.38	1.572	1.51

5.4 A step-size pathology the reduction cures

The `warn` column of Table 6 is not cosmetic. The legacy path emits, on every substep of every point,

```
[WARNING] Internal t = 9.88525e+06 and h = 2.10129e-12 are such that t + h = t on the
next step
```

2048 of them in a 1024-point, 5-substep march.

The cause is the accumulators. Each substep restarts at $t \approx 10^7$ s with the accumulators reset to zero, and with $\text{atol} = 10^{-20}$ their error weights $w_i = 1/(\text{rtol}|y_i| + \text{atol})$ are $\sim 10^{20}$. CVODE’s initial-step heuristic uses the weighted norm of $\dot{\mathbf{y}}$; with $\dot{a} \sim 10^{-7}$ this gives $\|\dot{\mathbf{y}}\|_{\text{WRMS}} \sim 10^{13}$ and hence $h_0 \sim 10^{-13}$, while $\varepsilon t \approx 2 \times 10^{-10}$. The first step is below the representable resolution of t and is wasted.

Removing the accumulators from the error test eliminates this outright: zero warnings, and 18% fewer internal steps. This is the reduction’s most defensible benefit — it is a robustness property, independent of problem size, whereas the LU saving is not.

A second, subtler effect works the other way. The error test is a root-mean square over components, $\|e\|_{\text{WRMS}} = \sqrt{N^{-1} \sum_i (e_i w_i)^2}$, so removing six well-behaved components makes the norm $\sqrt{19/9} = 1.45$ times more sensitive per remaining component. The 19-equation system was being flattered by six dummy components diluting its error norm; the reduced system is held to a genuinely tighter standard at the same nominal tolerance.

5.5 The quadrature memo, and why the reduction does not win on time

CVODES calls the quadrature right-hand side once per completed step. Because that call needs the same core evaluation the residual has just performed, the implementation memoizes the accumulator rates on the exact $(t, \mathbf{y})$ bit pattern and reuses them on a match.

The measured hit rate is poor: 1 hit against 2564 misses in the standalone run. CVODE’s corrector evaluates the residual at its Newton iterates, and the accepted step point $\mathbf{y}_n$ is generally not one of them, so the memo misses and a second full core sweep is paid per step. That is exactly the +28% in n_{core} visible in Table 4, and it cancels the LU saving.

The identified remedy, not implemented here, is to keep the accumulators in the state vector but neutralize their error weights with a vector `atol` (`CVodeSVtolerances` with $\text{atol} = 10^{300}$ on those six components). That would retain the step-size cure of Section 5.4 at *zero* extra core evaluations, at the cost of a 15×15 rather than 9×9 block — which, on this evidence, is the better trade. It is recorded as the next step rather than claimed as a result.

6 The 0-D $\leftrightarrow$ 3-D field handoff

6.1 What was wrong

The previous 0-D $\rightarrow$ 3-D carrier (`modelib_fem.write_sink_field`) wrote four scalars per dose step — one number density and one mean radius per loop family — into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt`. Those files drive MoDELlib’s *microstructure generator*, which places discrete loops at a uniform target density. The driver therefore had to collapse the marched state to its volume average before handing it over:

```
A = np.mean(np.asarray(Y), axis=0)
```

Every bit of spatial structure the slow march produced was discarded on the way out. In the reverse direction the mobile field was read from `F/quadrature_*.txt` through a column map whose source comment conceded it had never been checked against a real run.

6.2 What MoDELlib3 actually offers

MoDELlib3 does not need a scalar. `ClusterDynamicsFEM` already carries the immobile population as a finite-element *field* — `immobileClusters`, with `iSize = 8` components per node — and `ImmobileSinks` builds the sink strength from the local value at every quadrature point. `ClusterDynamics::output` writes that field, together with the `mSize = 4` mobile components, into the CD block of `evl/evl_<N>.txt`. Decisively, `ClusterDynamicsFEM::initializeConfiguration` reads it back:

```
if(size_t(configIO.cdMatrix().size())==mobileClusters.gSize()+immobileClusters.gSize())
{
    mobileClusters = configIO.cdMatrix().block(0,0,nNodes,mSize)...
    immobileClusters = configIO.cdMatrix().block(0,mSize,nNodes,iSize)...
}
```

The CD block is therefore a read/write field channel in both directions, on the node set the two codes already share. `evl/cdNodes.txt` gives the node coordinates in the same row order, so no search or interpolation is required.

6.3 The mapping

Table 8: CD block layout, `ClusterDynamicsParameters`: `mSize = 4`, `iSize = 8`. `immobileSpeciesVector = -1 1 1 1` makes family 0 the basal $\langle c \rangle$ vacancy family and families 1–3 the three prism $\langle a \rangle$ variants.

Columns	Block	Components	Units
0–3	mobile	C_v, C_i, C_{2i}, C_{3i}	atom fraction
4–7	immobile	$n_c, n_{a1}, n_{a2}, n_{a3}$	per b^3
8–11	immobile	$c_c, c_{a1}, c_{a2}, c_{a3}$	atom fraction

MoDELlib works in Burgers-vector units, so its nodal number density is per b^3 while the 0-D loop density is an atom fraction. `ImmobileSinks` floors them as $n_{\text{floor}} = C_{\text{floor}}/\omega$ and $c_{\text{floor}} = C_{\text{floor}}$, which fixes the conversion:

$$n_{\text{MoDELlib}} = \frac{C_{0D}}{\omega}, \quad c_{\text{MoDELlib}} = c_{0D}, \quad \omega = \frac{\text{atomicVolume_SI}}{b_SI^3} = 0.355111 \, b^3. \quad (2)$$

The 0-D aligned/non-aligned pair is collapsed and the $\langle a \rangle$ total distributed over the three prism variants by the resolved-stress weights — the same Option-A reduction `modelib_export` performs, but applied *pointwise* rather than to a volume average. The mobile direction is a pure slice: MoDELlib’s four mobile species *are* the 0-D mobile species, in the same order and units.

6.4 Verification on the 3-D verification case

Tested against MoDELlib3/tutorials/zrmicro_coupled, the 1 μm single-crystal case with 24 115 CD nodes, at `evl_30.txt`.

Table 9: Field-handoff verification, 24 115 CD nodes.

Check	Result
CD block rewrite round-trip	identical, $24\,115 \times 12$; header and all dislocation records preserved
0-D $\rightarrow$ MoDELlib $\rightarrow$ 0-D immobile	max relative error 4.47×10^{-16} (machine precision)
Mobile field read back	C_v spans 8.46×10^{-15} to 6.70×10^{-6} , a dynamic range of 7.9×10^8
Spatial structure	$\text{corr}(\log_{10} C_v, \text{distance to nearest face}) = +0.63$
March on the real node set	24 115 nodes over 1 dpa in 9.5 s, 0 failures
Field written back	n_{a1} spans a factor 4.4 across the domain

The information content of the handoff rises from 4 numbers to $24\,115 \times 12 = 289\,380$ numbers, and the boundary-layer structure that the Dirichlet condition imposes on the mobile field — eight decades in C_v , correlating with distance to the surface — now reaches the slow march instead of being averaged away.

A precision detail. Restoring the aligned/non-aligned split on the return trip initially lost eight significant digits on the minor partner. The fractions were formed as f and $1 - f$; when the pair is lopsided (and $C_{iL} \gg C_{aiL}$ is normal at low dose) f rounds to $1 - \epsilon$ and $1 - f$ retains only the bits surviving the cancellation. Forming both fractions directly from the pair restores full precision, giving the 4.47×10^{-16} of Table 9.

6.5 Notebook wiring

`coupled_0d_3d_ZrMicro.ipynb` gains a "field" fast-solve backend, selected automatically when a CD field is present. In that mode the march runs on the CD node set rather than the synthetic 1-D line, seeds the immobile state from the 3-D field itself, and after each dose interval writes `evl/evl_coupled_<dose>dpa.txt` — a complete configuration, with the dislocation records copied through unchanged, that MoDELlib can be restarted from. The previous scalar path and the analytic placeholder both remain available and unmodified.

Because MoDELlib reads its configuration at start-up, the handback takes effect on a *restart* from the written file. That is the rhythm the operator split requires anyway (fast solve $\rightarrow$ slow march $\rightarrow$ fast solve), but it does mean the coupling is restart-driven rather than in-process: at the default six snapshots that is six DDomp launches, which is unremarkable; at hundreds of dose intervals it would argue for driving MoDELlib through `pyMoDELlib` instead.

6.6 Measured cost of the field upgrade

The comparison needs care, because the old path never ran on the mesh at all. `RUN["n_points"] = 25`: it marched a synthetic 1-D line of 25 points bearing no relation to the geometry, while the field path marches the 24 115 CD nodes — 965 times as many. A same-work control is therefore included: the git-HEAD solver driven over the *same* 24 115 nodes, which is what the old solver would have cost to do the new job.

Test. The notebook's default schedule — 1 to 26 dpa, five intervals, `substeps_per_interval = 20`, i.e. 100 batch subprocesses — run to completion in all three configurations, seeded at 1 dpa from the 0-D reference and, for the field configurations, from the CD field itself.

Table 10: Full 1–26 dpa coupling march, 100 batch subprocesses, 24 cores.

	Configuration	q	wall [s]	s/substep	ms/point/substep
A	Old as configured (git HEAD, 1-D line)	25	4.3	0.04	1.715
B	Old solver at the new workload	24 115	1117.4	11.17	0.463
C	New: $9 \times 9 + \text{AD} + \text{reuse}$	24 115	574.2	5.74	0.238

Two distinct readings, and both are needed to avoid misrepresenting the change:

- **Same work, B versus C:** $1.95\times$ **faster**. That is the honest solver speedup — identical points, identical schedule. It exceeds the $1.67\times$ measured at $q = 1024$ in Table 7 because the per-case solver construction the old binary pays scales with q while the new binary’s per-thread construction does not: 24 115 solver builds per substep become 24.
- **Different work, A versus C:** $134\times$ **more wall clock for** $965\times$ **the points**. The old configuration is faster in absolute terms only because it was doing $1/965$ of the work and emitting eight numbers. Per point per substep the new path is $7.2\times$ cheaper ($1.715 \rightarrow 0.238$ ms), part solver improvement and part fixed process-and-parse overhead amortizing over far more cases.

For scale: the 3-D solve itself (`clean_run.sh` $\rightarrow$ DDomp) takes hours. Against that, neither 4 s nor 10 min makes the march the pacing item of a coupled run.

Consistency. Over the full 1–26 dpa march, configurations B and C agree to 9.05×10^{-7} in the species. That is *tighter* than the 8.2×10^{-4} measured after the first 5 dpa interval alone, and the direction is informative: the early transient difference between two step sequences washes out rather than accumulating, because the loop populations are approaching a quasi-steady state and the dynamics are attracting. The largest disagreements sit at the stiff near-boundary nodes, where the frozen mobile field is eight decades below the interior value. It is a tolerance effect, not a change of answer.

7 Integrator comparison: MoDELib’s nodal update versus the CVODE march

The two codes advance the *same* immobile physics — eight components per node — with entirely different schemes. This section characterizes both, since the difference is a systematic source of 0-D/3-D discrepancy that is independent of the physics non-correspondences already documented in `CLAUDE.md`.

7.1 What MoDELib actually does

It is worth stating precisely, because the scheme is often described as explicit Euler and it is not. `ClusterDynamicsFEM::solveImmobileClusters` loops over nodes and, inside each node, over `nSub = 20` substeps of $dt = dtMax/20$, applying

```
// implicit for the linear losses, explicit for the sources
const double nNew((n(k)+dt*nucRate)/(1.0+dt*lossN));
const double cNew((c(k)+dt*(gdot(k)+Gk+clusShare(k)*clusC(pol)-contentSink))/(1.0+dt*lossC));
```

This is a first-order **semi-implicit (IMEX)** update: the sources (nucleation, absorbed flux, cascade seed) are explicit, while the linear loss coefficients — coalescence, $1/\tau_{vL}$, and the thermal-emission rate recast as a first-order coefficient — sit in the denominator. The source comment

records the reason: “ τ_{vL} is short compared with a dose step ...an explicit update would give $n(1 - dt/\tau) < 0$ and collapse the vacancy-loop density.”

With $G = 10^{-7}$ dpa/s and $dt_{\text{Max}} = 6.95868964\text{e}19$ in MoDELib units (time unit 1.4371×10^{-13} s), one dose step is 10^7 s and the immobile substep is

$$dt_{\text{MoDELib}} = \frac{10^7 \text{ s}}{20} = 5 \times 10^5 \text{ s.} \quad (3)$$

7.2 Test method

Three instruments, all exercising the *same* `rate_equations_core.h` right-hand side so that no model difference contaminates the comparison:

1. **Spectrum.** The frozen-mobile 9×9 immobile Jacobian is formed at states sampled along a converged trajectory and its eigenvalues taken. The forward-Euler stability limit is $dt < 2/|\text{Re } \lambda|_{\text{max}}$.
2. **Fixed-step explicit reference.** A `-euler_nsub=N` mode was added to the solver: N forward-Euler substeps per output interval on the same core, with the per-substep positivity clamp MoDELib applies. This is the *control* — what the update would do without the implicit loss treatment — not a model of MoDELib’s scheme.
3. **Reference solution.** CVODE at `rtol=1e-12`, `atol=1e-26` with the AD Jacobian, against which every other result is measured.

Two nodes of the 1 μm case bracket the range: the interior node of maximum C_v and the near-boundary node of minimum C_v .

7.3 Stability

Table 11: Spectrum of the frozen-mobile 9×9 immobile Jacobian.

Dose [dpa]	$ \text{Re } \lambda _{\text{max}} [\text{s}^{-1}]$	$dt_{\text{stab}} = 2/ \text{Re } \lambda _{\text{max}}$	stiffness ratio	$dt_{\text{MoDELib}}/dt_{\text{stab}}$
0.99	1.854×10^{-1}	10.8 s	1.5×10^5	46 338
2.96	1.853×10^{-1}	10.8 s	5.7×10^7	46 335
5.91	1.853×10^{-1}	10.8 s	4.0×10^5	46 335
11.16	1.853×10^{-1}	10.8 s	1.2×10^5	46 335
15.78	1.853×10^{-1}	10.8 s	3.1×10^4	46 335
21.05	1.853×10^{-1}	10.8 s	1.8×10^7	46 335
26.52	1.853×10^{-1}	10.8 s	9.9×10^4	46 335

The fast eigenvalue is essentially dose-independent at 0.185 s^{-1} — the vacancy-loop dissolution channel, $n_{vL,\text{nuc}}/\tau_{vL}$ — so a forward-Euler step must satisfy $dt < 10.8 \text{ s}$. MoDELib runs at $5 \times 10^5 \text{ s}$, **46 000 times the explicit limit**. Its implicit loss treatment is therefore not a refinement; it is what makes the scheme usable at all.

Table 12 shows the consequence directly, from the fixed-step explicit control at MoDELib’s own substep count.

Both schemes actually in use are unconditionally stable on the stiff channel, but by different mechanisms. MoDELib’s amplification factor $1/(1 + dt \lambda) \in (0, 1)$ for every $dt > 0$ and $\lambda > 0$: L-stable on the linear part, and positivity-preserving by construction. CVODE BDF is A-stable at orders 1–2 and stiffly stable to order 5, and in addition selects its own step from a local error estimate.

Table 12: One 1 dpa step at the near-boundary node: a genuine forward Euler at $dt = 5 \times 10^5$ s annihilates the entire $\langle c \rangle$ vacancy-loop family, exactly the failure the MoDELib source comment anticipates. The $\langle a \rangle$ families, which carry no fast decay channel, are unaffected.

Species	initial	CVODE reference	explicit Euler, $N = 20$	rel. error
C_{iL}	2.53×10^{-3}	5.69×10^{-3}	5.69×10^{-3}	1.8×10^{-11}
C_{aiL}	1.60×10^{-3}	3.44×10^{-3}	3.44×10^{-3}	2.9×10^{-11}
$C_{iL,i}$	2.16×10^{-1}	2.79×10^{-1}	2.79×10^{-1}	2.2×10^{-10}
$C_{aiL,i}$	1.37×10^{-1}	1.73×10^{-1}	1.73×10^{-1}	2.3×10^{-10}
C_{vL}	3.48×10^{-10}	3.48×10^{-10}	1.0×10^{-20} (floor)	1.00
C_{avL}	2.02×10^{-10}	2.02×10^{-10}	1.0×10^{-20} (floor)	1.00
$C_{vL,v}$	7.21×10^{-4}	7.21×10^{-4}	1.0×10^{-20} (floor)	1.00
$C_{avL,v}$	4.20×10^{-4}	4.20×10^{-4}	1.0×10^{-20} (floor)	1.00

Table 13: One 1 dpa dose step, interior node ($C_v = 6.70 \times 10^{-6}$), against the `rtol=1e-12` reference. “core evals” counts evaluations of the shared right-hand side.

Integrator	substeps	dt [s]	core evals	max rel. error	t [ms]
Explicit Euler (MoDELib’s <code>nSub</code>)	20	5.0×10^5	20	9.27×10^4	0.14
Explicit Euler	200	5.0×10^4	200	9.27×10^3	0.27
Explicit Euler	2000	5.0×10^3	2000	9.26×10^2	1.18
Explicit Euler	20000	5.0×10^2	20000	9.17×10^1	13.71
CVODE BDF 9 × 9 + AD, <code>rtol=1e-6</code>	161 (adaptive)	6.2×10^4	366	2.10×10^{-6}	1.50

7.4 Accuracy and speed

At the near-boundary node CVODE needs only 37 steps and 84 core evaluations for 1.27×10^{-8} , the frozen mobile field there being effectively zero.

The explicit control converges cleanly at first order — a factor 10 in error per factor 10 in step count — which confirms the order; the enormous constant reflects that it is simultaneously unstable.

What can and cannot be claimed for MoDELib’s scheme. Its IMEX splitting was not reimplemented on the ZrMicro equations, so no error *number* is quoted for it here. What is structural, and visible in the code itself, is that it is **first-order accurate with no error control**: no local error estimate, no step rejection, no adaptivity, and 20 fixed substeps per dose step whatever the local stiffness. Against that, CVODE selects its own order (up to 5) and step from a local error estimate at `rtol` = 10^{-6} .

Speed. Per node per dose step, MoDELib performs 20 nodal updates of eight components with no Jacobian, no linear solve and no error estimate; the comparable explicit control costs 0.14 ms against CVODE’s 1.50 ms interior and 0.76 ms near the boundary. So MoDELib’s immobile update is roughly **an order of magnitude cheaper per node**, which is precisely the price of first order and no adaptivity. At the scale of the coupled march this is the difference between the 3-D solve’s nodal update being free and the 0-D march costing the 574 s of Table 10.

7.5 Consequence for the coupling

The operator split is itself first order in $\Delta\gamma$ (Section 7 notwithstanding, this is the scheme’s design order). MoDELib’s immobile update is therefore *consistent with* the splitting order, and

Table 14: The two integrators side by side.

	MoDELib solveImmobileClusters	ZrMicro coupled march
Scheme	semi-implicit (IMEX), nodal	CVODE BDF, adaptive
Order	1	up to 5
Step	fixed, 5×10^5 s (20/dpa)	adaptive (~ 161 /dpa interior)
Error control	none	local estimate, $\text{rtol} = 10^{-6}$
Stability	L-stable on the linear losses	stiffly stable to order 5
Linear algebra	none (diagonal by construction)	dense 9×9 LU, exact AD Jacobian
Cost per node	~ 0.14 ms / dpa	~ 0.8 – 1.5 ms / dpa
Accuracy	first order, unquantified	2×10^{-6} measured

CVODE’s six additional digits on the slow march buy nothing unless $\Delta\gamma$ is refined alongside them.

Where the difference does matter is the return path. Once the marched field is written back through the CD block (Section 6.6), the two schemes’ different truncation errors become a systematic 0-D/3-D discrepancy — a *fourth* candidate alongside the three physics non-correspondences listed in CLAUDE.md (the DAD bias reconciliation, the bi-pyramid vacancy family, and size-dependent vacancy-loop emission). Anyone chasing a residual disagreement between the interior 0-D state and the 3-D result should rule this one out first, because it is the cheapest to test: raise `nSub` in `solveImmobileClusters` and see whether the disagreement moves.

8 Provenance: how the two schemes came to be

Section 7 compared the two integrators as they stand. This section records where each came from, because the histories explain the asymmetry between them and carry a maintenance consequence.

Table 15: Immobile-species integration across the three code states.

	MoDELib2-NNL (upstream)	MoDELib3 (standalone route)	ZrMicro coupled march (this work)
iSize (immobile dofs/node)	0	8	—
Immobile integrator	empty stub	semi-implicit, nodal	CVODE BDF
ImmobileSinks.h	absent	new file	n/a
Order	—	1	up to 5
Step control	—	fixed, 20/dpa	adaptive
Error control	—	none	$\text{rtol} = 10^{-6}$
Loop sink on the mobile	none	bWF_RI/lWF_RI	frozen $\mathbf{C}_M^*$

8.1 The standalone scheme: from absent, to explicit, to semi-implicit

There is no upstream scheme to compare against. In MoDELib2-NNL the immobile machinery does not exist: `iSize` is zero, `solveImmobileClusters()` is an empty stub, `ImmobileSinks.h` is absent, and the mobile species see only the network sink through `otherSinks_SI`, the source carrying a `// Missing immobile sinks` placeholder where the loop term would go. The material files corroborate this independently: `Zr4.txt`, the upstream-era standalone 3-D calibration, contains **zero** immobile keys (`immobileSpecies*`, `tauVac`, `nNuc`, `cLL`, `kappaLL`, `dad*`) against eight such key families in `Zr3d_ghoniem.txt` — which is exactly why `Zr4.txt` is incompatible with the fork. Consequently the upstream standalone route cannot produce an $\langle a \rangle / \langle c \rangle$ loop microstructure at all; the 1 μm case requires MoDELib3.

MoDELib3 introduced the field and its integrator wholesale — but not in its present form. It began fully explicit, and reached the semi-implicit update through four documented corrections, recorded in `MoDELib3/ZR3D_GHONIEM_CHANGES.md`:

Table 16: How the standalone immobile scheme reached its present form. Each stiffness failure was fixed by moving one more linear loss into the implicit denominator.

Issue	Symptom	Cause	Fix
7	$\langle c \rangle$ loops pinned on the positivity floor while $\langle a \rangle$ loops nucleated normally	τ_{vL} is $35\times$ shorter than the substep, so explicit Euler gave $n(1 - 35) < 0$, clamped every iteration	Integrate the linear losses implicitly; returns $n^* = \dot{n}_{\text{nuc}}\tau_{vL}$ exactly as $dt/\tau \rightarrow \infty$
8	Thermal emission could not be made implicit	It was written as an absolute rate, not proportional to the stored content	Recast as a first-order coefficient (divide by c) so it can enter the implicit update
14	$\langle a \rangle$ mean size 5042 at 1 dpa, then exactly $300.0 = n_{\text{nuc}}$ at 2 dpa — content collapsing $11\times$	The same stiffness failure in the coalescence channel left explicit: $\nu_{LN}\phi_{LN}dt_{\text{sub}} \approx 3.6$	Eq. (97) removes n and c proportionally, so both are first-order coefficients; moved implicit. $\langle a \rangle$ density went from $3\text{--}4\times$ the 0-D to within $\sim 30\%$
15	$\langle c \rangle$ content $350\times$ below the 0-D, mean size pinned at the nucleation size	Dissolution removed content proportional to the stored content	Release the birth content $n_{\text{nuc}}n/\tau_{vL}$ instead

Issue 7 is worth singling out because its diagnosis was made the right way: the authors confirmed it was integration and not physics by checking the analytic balance $n^* = \dot{n}_{\text{nuc}}\tau_{vL} = 1.79 \times 10^{-8}$ against the 0-D $C_{vL} + C_{avL} = 1.78 \times 10^{-8}$, then verified the fix reproduced 1.79161×10^{-8} .

8.2 The coupled march scheme: outward from the 0-D solver

The march scheme has the opposite history. It did not have to discover stiffness, because it inherited a stiff integrator: ZrMicro’s CVODE BDF solver predates the coupling entirely, built for the standalone 0-D 19-equation model. The coupling was then added in three steps:

1. **freeze_mobile** — the operator split. The four mobile derivatives are zeroed *last*, after every reaction term has already read them as parameters, so the same right-hand side serves as the local immobile micro-model with an externally supplied $\mathbf{C}_M^*$.
2. **Pointwise dispatch** — because the split makes the slow march decoupled between points, each quadrature point becomes one case of the existing OpenMP `-batch_file` mode.
3. **This work** — accumulators moved to CVODES quadrature (19 $\rightarrow$ 9), the exact AD Jacobian, and per-thread workspace reuse.

8.3 Two routes to the same conclusion

Both codes ended up treating the immobile loss channels implicitly, and both were right to, but they arrived from opposite directions. MoDELib3 found the stiffness empirically — twice, as loop

populations collapsing onto the positivity floor — and fixed it term by term. The 0-D side never exposed the problem, because BDF absorbed it silently from the start; it took the spectrum of Table 11 to put a number on what was being absorbed ($46\,000\times$ the forward-Euler limit).

That difference in route leaves a difference in robustness, and it is the practical point of this section. MoDELib3’s scheme is unconditionally stable on the losses that *have been* moved into `lossN/lossC`. It offers no protection for one that has not. A newly added immobile loss term will be stable only if it is first recast as a first-order coefficient and placed in the denominator; written as an explicit rate it will reproduce issue 7 or issue 14, and — with no local error estimate and no step rejection — the only symptom will again be a species quietly pinned on the floor. CVODE would have flagged the same term through step rejection and error-test failures without anyone having to notice a collapsed population.

Sourcing. MoDELib2-NNL is not present in this checkout: there is no vendored copy, and MoDELib3/ carries no independent git history, having been flattened from a submodule (the six commits present are DisloCluster’s own). The upstream statements above therefore rest on MoDELib3’s own change log — a detailed first-party record — corroborated by the direct evidence of the `Zr4.txt` versus `Zr3d_ghoniem.txt` key comparison, which was checked against the files in this repository. They have not been verified against the upstream repository itself.

9 Summary and recommended next steps

Table 17: Summary of outcomes against intent.

Change	Target metric	Outcome
AD Jacobian	residual evaluations	−33%; wall neutral
9×9 reduction	LU flops	−92%; steps −18%; wall −4%
	step-size pathology	2048 roundoff warnings $\rightarrow$ 0
Workspace reuse	solver setup cost	march wall $\times 1.67$ at $q = 1024$
Field handoff	information exchanged	4 scalars $\rightarrow$ 289 380 values
Full 1–26 dpa march	same-work solver speed	$1.95\times$ (Table 10)

Two findings are worth carrying forward independently of the changes themselves. First, the accumulators were quietly setting the initial step of every coupling substep, and removing them from the error test cures a roundoff-limited first step that had been occurring 2048 times per march (Section 5.4). Second, the two codes integrate the same immobile equations to very different error levels — first order with no error control on the 3-D side, adaptive to 10^{-6} on the 0-D side — which is a systematic 0-D/3-D discrepancy distinct from the three physics non-correspondences on record (Section 7).

Ordered by expected return:

1. **Replace the quadrature with a vector-`atol` state** (Section 5.5). Keeps the step-size cure at zero extra core evaluations; should convert the reduction’s current −4% into a clear win.
2. **Replace the text case-file protocol.** Every batch line repeats the full material parameter set; only the 19 `y0` values differ. At $q = 1024$ that is $\sim 1.5 \times 10^5$ redundant string-to-double conversions per substep. A base-plus-delta or binary format would attack what is now, after the workspace change, the largest remaining non-physics cost.
3. **Deduplicate quadrature points before dispatch.** In a smooth field neighbouring nodes integrate nearly identical initial-value problems. Clustering the 13-dimensional input state

to a tolerance could collapse $q \sim 10^5$ to 10^2 – 10^3 genuine integrations — likely a larger factor than everything above combined, and it composes with all of it.

4. **Guard the standalone immobile solver against the next stiff term.** Section 8 shows the same failure was found twice, by symptom, and there is still no mechanism that would catch a third. Any new loss channel added to `solveImmobileClusters` must be recast as a first-order coefficient and placed in `lossN/lossC`; a cheap standing check is to run one case at `nSub` and at `10 nSub` and assert the answers agree, which turns a silent floor collapse into a visible regression.
5. **Verify the 3-D fast solve end to end** in the field backend, then revisit GPU offload. Until the spatial solve actually runs inside the march loop, q is set by the node count of a stored configuration rather than by a live coupling, and the case for batched GPU integration cannot be assessed on real numbers.

Files. `cpp_utils/dual.h`, `cpp_utils/rate_equations_core.h`, `cpp_utils/rate_equations.{h,cpp}`, `cpp_utils/jac_check.cpp`, `cpp_utils/parameters.h`, `code/solver.cpp`, `py_utils/cpp_bridge.py`, `py_utils/modelib_coupling.py`, `py_utils/modelib_field.py` (new), `code/coupled_0d_3d_ZrMicro.ipynb`. The build now links `SUNDIALS::cvodes` rather than `SUNDIALS::cvsode`, because `CVodeQuadInit` and its companions exist only in CVODES; the header `<cvodes/cvodes.h>` was already being included.